

PRODUCTION EDITION V3.0

AI Operating System Jarvis-Style Blueprint

Local-First | Memory-Driven | Security-Gated | Human-Supervised

AUTHOR Swapnil — Agent Engineer

YEAR 2026

CLASSIFICATION Technical Architecture Document

ENHANCED BY CLAUDE (ANTHROPIC) • CONFIDENTIAL
PORTFOLIO DOCUMENT

Table of Contents

0. Executive Summary

1. Project Vision & Current Position

1.1 Architecture Paradigms

1.2 Implementation Status Matrix

2. Industry Benchmarking & Competitive Analysis

2.1 State-of-the-Art Comparison

2.2 Differentiation Strategy

3. Target Architecture — Execution Pipeline

3.1 Pipeline Diagram

3.2 Stage Specifications

3.3 Component Interaction Matrix

4. Memory Architecture & Mathematical Retrieval

4.1 Four-Layer Storage Model

4.2 Hybrid Retrieval & Decay Scoring

4.3 Contradiction Detection Flow

4.4 Memory Latency Budget

5. Deterministic Security Model

5.1 Three-Zone Classification

5.2 Advanced Guardrail Controls

5.3 Failure Mode & Effects Analysis

6. Core System Engine Specifications

6.1 Mistake Database Schema

6.2 Cryptographic Audit Log

6.3 API Contract Specifications

7. Testing & Verification Strategy

7.1 Testing Pyramid

7.2 Security Testing Protocol

7.3 Chaos Engineering Plan

8. Deployment Architecture & Observability

8.1 Deployment Topology

8.2 Performance & Scalability Specifications

8.3 Monitoring & Alerting Framework

9. Trust Principles for Industrial Development

10. Risk Assessment Matrix

11. Engineering Skills Matrix

12. Recruiter Showcase & Evaluation Scenario

12.1 Demo Script Timeline

12.2 Talking Points by Competency

13. Development Roadmap

References

0. Executive Summary

This document presents the production-grade architecture blueprint for a local-first AI Operating System modeled on the Jarvis paradigm — a supervised, memory-driven, security-gated agentic system that maintains human authority at every critical decision point. The system integrates multi-layered episodic and semantic memory with deterministic cryptographic guardrails, automated reflection-based self-correction, and tamper-evident audit logging.

The architecture advances beyond conventional chatbot patterns by implementing a full **execution pipeline** spanning voice input, planning, hybrid memory retrieval, security classification, human approval, sandboxed execution, verification, reflection, and cryptographic audit — with no component permitted to bypass the Security Gateway. The blueprint synthesizes 2025–2026 state-of-the-art patterns from production agent systems including OpenAI's Operator-class architectures, Anthropic's computer use frameworks, and MemGPT/Letta memory management research.

Key differentiators: (1) deterministic fail-closed security zoning with scope-locking and secret scanning; (2) four-layer memory architecture with hybrid BM25+FAISS retrieval and temporal decay scoring; (3) cryptographic SHA-256 hash-chained append-only audit logging providing tamper-evidence guarantees; (4) automated reflection with structured mistake database enabling cross-session learning; and (5) comprehensive human-in-the-loop authority at every critical execution gate with confidence-threshold gating.

The system targets **40–50% implementation completion** with a focused MVP scope delivering a demonstrable two-minute recruiter showcase highlighting memory replay, security gate validation, reflection logging, and cryptographic audit integrity verification. This document serves as both a technical architecture specification and a portfolio artifact demonstrating enterprise-grade system design capability across security engineering, distributed systems architecture, database design, and DevOps operations.

1. Project Vision & Current Position

1.1 Architecture Paradigms

The AI Operating System is constructed upon four foundational paradigms that collectively define its design philosophy and operational guarantees. The **local-first paradigm** mandates that all inference, memory retrieval, and security classification execute on local hardware without dependency on external network services, ensuring data sovereignty and eliminating latency variability from cloud round-trips. This choice carries inherent trade-offs: model capability is bounded by local GPU/CPU constraints, and vector search

throughput is limited to single-node performance. However, for a personal AI OS operating on sensitive codebases and credentials, local-first is a non-negotiable security posture.

The **memory-driven paradigm** elevates persistent structured memory from a caching convenience to a core architectural primitive. Unlike stateless LLM APIs that treat each interaction as independent, this system maintains four distinct memory layers — working, episodic, semantic, and mistake memory — each with specialized retrieval patterns, persistence guarantees, and TTL policies. The memory architecture enables cross-session continuity: the agent remembers not only what was done, but why previous approaches failed, creating a compounding knowledge base that improves agent performance over deployment lifetime.

The **security-gated paradigm** establishes that no component — including the Planner Agent, Executor Agent, or Reflection Agent — can directly modify the host system without deterministic classification and explicit human approval. The Security Gateway operates as an unbypassable kernel module that classifies every proposed action into Green (safe), Yellow (caution), or Red (danger) zones, with escalating approval requirements. The gateway follows a fail-closed policy: ambiguous classifications default to the most restrictive zone, never permissive.

The **human-supervised paradigm** embeds human authority as a first-class architectural constraint rather than an optional override. Every execution pipeline stage includes explicit human decision points for operations classified as Yellow or Red zone, with confidence score gating (threshold: 0.72) providing an additional layer of automated escalation independent of security classification. The system is designed to propose, never to autonomously execute critical operations.

1.2 Implementation Status Matrix

Table 1 Implementation Status Matrix — Core Capabilities

Capability	Status	Priority	Technical Notes
LLM-Powered Code Generation	LIVE	P0	Multi-model routing (Ollama, local GGUF); temperature scaling per task type
Live Preview Environment	LIVE	P0	Hot-reload via file watcher; WebSocket state sync to UI
Terminal Interaction	LIVE	P0	Sandboxed subprocess with restricted env; stdout/stderr capture
File Modification	LIVE	P0	Human-in-the-loop diff preview; git-aware change tracking
User Approval Mechanism	LIVE	P0	Non-blocking UI prompts; typed confirmation for RED zone
SQLite Memory Layer	LIVE	P0	Episodic & semantic schemas operational; WAL mode enabled
Deterministic Security Gateway	ACTIVE	P1	Zone classifier in development; fail-closed logic implemented
Vector Memory (FAISS)	ACTIVE	P1	Hybrid search index with HNSW; 384-dim embeddings (all-MiniLM-L6-v2)
Reflection Agent & Mistake DB	NEXT	P1	Schema finalized; post-task root-cause analysis layer pending
Rollback Engine	PLANNED	P2	Git-stash integration; file-level snapshot with SHA verification
Voice Interface	PLANNED	P2	Whisper STT + Piper TTS; wake-word detection; fully offline
Project Knowledge Graph	PLANNED	P3	Neo4j entity-relationship graph; multi-hop codebase reasoning

2. Industry Benchmarking & Competitive Analysis

2.1 State-of-the-Art Comparison

As of 2026, the agentic AI landscape has matured significantly from the experimental phase of 2023–2024. Understanding where this architecture positions against production systems is essential for identifying genuine differentiation versus table-stakes features. The following analysis compares the AI OS blueprint against four representative systems spanning commercial, open-source, and research categories.

Table 2 Competitive Analysis — Agentic AI Systems (2026)

Dimension	AI OS (This Blueprint)	OpenAI Operator	Anthropic Computer Use	AutoGPT / Agent Zero	Letta (MemGPT)
Deployment Model	Local-first, fully offline	Cloud-dependent, API-gated	Cloud-dependent, API-gated	Hybrid (local + cloud)	Self-hosted or cloud
Memory Architecture	4-layer (working, episodic, semantic, mistake)	Conversation thread only	Context window limited	Vector DB (single layer)	3-tier (core, archival, recall)
Security Model	Deterministic 3-zone with fail-closed	Usage policy filter	Computer use safeguards	User-configurable	Function-level control
Human Oversight	Required at every critical gate	Optional confirmation	Screenshot-based review	Optional approval	Human-in-the-loop tool
Cryptographic Audit	SHA-256 hash-chained, tamper-evident	Not exposed	Not exposed	Standard logging	Standard logging
Reflection / Learning	Structured mistake DB with confidence delta	Not applicable	Not applicable	Basic error logging	Memory management functions
Codebase Understanding	Project knowledge graph (planned)	N/A	N/A	Limited file access	N/A
Voice Interface	Whisper + Piper, offline	Cloud TTS/STT	Text only	Plugin available	Text only
Rollback Capability	Git-stash + file snapshot	N/A	N/A	Git integration	N/A

2.2 Differentiation Strategy

The AI OS blueprint occupies a distinctive position: it is the only architecture in this comparison that combines **local-first deployment** with **multi-layered persistent memory**, **deterministic security gating**, and **cryptographic tamper-evident audit logging** within a unified, human-supervised execution pipeline. Where commercial systems prioritize capability breadth and cloud scale, this blueprint prioritizes **sovereignty, explainability, and verifiability** — qualities essential for developer tooling that operates on proprietary source code and credentials.

The Letta (MemGPT) architecture represents the closest academic parallel, particularly in its three-tier memory management model. However, Letta focuses on conversation context management rather than code manipulation, and lacks the deterministic security zoning and rollback capabilities central to this blueprint. The recommendation is to study Letta's memory paging algorithms (`core_memory_append`, `archival_memory_search`) as a reference for optimizing the Working Memory to Episodic Memory transition, while maintaining the unique security and audit features as primary differentiators.

3. Target Architecture — Execution Pipeline

3.1 Pipeline Diagram

The execution pipeline enforces a strict linear flow through validation filters before any action touches the host system. No component — including internal agents — can bypass the Security Gateway. The pipeline comprises ten stages from input ingestion to audit logging, with feedback loops for reflection and rollback.

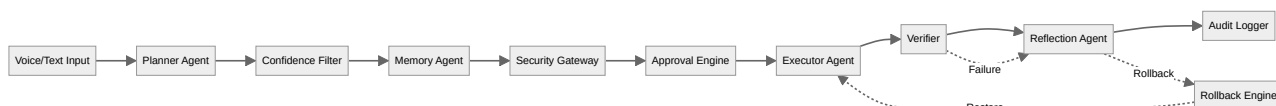


Figure 1: End-to-End Execution Pipeline with Feedback Loops

3.2 Stage Specifications

Table 3 Pipeline Stage Specifications

Stage	Responsibility	Key Technology	Latency Budget
1. Voice/Text Interface	Wake-word detection, STT/TTS, noise suppression	Whisper (STT), Piper (TTS)	< 200ms (STT)
2. Planner Agent	Goal decomposition to sub-task tree; confidence scoring per step	Chain-of-thought prompting	< 2s per decomposition
3. Confidence Filter	Gating: steps below 0.72 auto-escalate to human review	Configurable threshold	< 10ms
4. Memory Agent	Hybrid retrieval from RAM, SQLite, FAISS with re-ranking	BM25 + FAISS HNSW	< 150ms
5. Security Gateway	Deterministic zone classification (G/Y/R)	Regex + vector blocklist	< 50ms
6. Approval Engine	Interactive UI prompts; diff preview; typed confirmation	Non-blocking WebSocket UI	Human-dependent
7. Executor Agent	Sandboxed subprocess execution; scope-locked file tools	Subprocess + chroot	Task-dependent
8. Verifier Layer	Test assertions (pytest/jest); delta computation	pytest, jest, diff	< 30s (test suite)
9. Reflection Agent	Post-task root-cause analysis; structured lesson extraction	LLM prompt + schema validation	< 5s
10. Audit Logger	Append-only SHA-256 hash-chained logging	SQLite + Python hashlib	< 20ms
11. Rollback Engine	Git-stash or file-snapshot restore; post-rollback verification	GitPython, shutil	< 5s

3.3 Component Interaction Matrix

Table 4 Component Interaction Matrix — API Contracts

Source Component	Target Component	Interface Type	Payload Schema	Failure Behavior
Planner Agent	Confidence Filter	Internal function call	TaskTree + ConfidenceScore[]	Reject if malformed; log to Audit
Confidence Filter	Human UI	WebSocket emit	EscalationRequest + Context	Queue with 60s timeout
Memory Agent	SQLite / FAISS	SQL + FAISS Python API	QueryVector + BM25Terms + TopK	Return empty set; do not fail
Security Gateway	Approval Engine	Internal event bus	ZoneClassification + PayloadHash	Fail-closed → RED zone
Approval Engine	Executor Agent	Conditional pass-through	ApprovedPayload + Timestamp + DecisionID	Timeout → auto-cancel
Executor Agent	Verifier Layer	Pipeline pass	ExecutionResult + stdout + stderr + exit_code	Mark failed; trigger Reflection
Reflection Agent	Mistake DB	SQL INSERT	MistakeSchema (validated)	Queue for retry; alert if persistent
Audit Logger	SQLite (append-only)	SQL INSERT	AuditEntry + SHA-256 hash	Fatal; halt pipeline

4. Memory Architecture & Mathematical Retrieval

4.1 Four-Layer Storage Model

The memory architecture implements a hierarchical storage model inspired by human cognitive memory systems and the MemGPT/Letta research lineage, adapted specifically for code manipulation and task-oriented agent workflows. Each layer serves a distinct purpose with matched storage backends, access patterns, and persistence guarantees.

Table 5 Four-Layer Memory Storage Specification

Layer	Backend	Contents	Access Pattern	TTL
Working Memory (L1)	RAM (Python dict)	Active task contexts, tool runtime vars, conversation history	Direct dict access; O(1)	Session termination
Episodic Memory (L2)	SQLite	Historical tasks, goals, executions, outcomes, timestamps	Time-range + semantic query	Permanent
Semantic Memory (L3)	SQLite	Project entities, user preferences, codebase facts	Entity-relation query	Permanent
Mistake Memory (L4)	SQLite	Errors, root causes, fixes, lessons, confidence deltas	Error-type + similarity query	Permanent
Vector Index	FAISS (HNSW)	Embeddings for episodic + semantic entries	Approximate nearest neighbor	Synced with SQLite

4.2 Hybrid Retrieval & Decay Scoring

To prevent context window pollution while maximizing retrieval relevance, the system employs a hybrid scoring function combining lexical keyword matching, vector semantic similarity, and temporal recency with exponential decay. This approach aligns with 2026 production best practices where pure vector search is supplemented with BM25 keyword matching to capture exact terms (error codes, function names, file paths) that semantic similarity may miss.

MEMORY RELEVANCE SCORE FUNCTION

Given a query q , candidate memory m , and current time t , the composite relevance score $R(q, m, t)$ is defined as:

$$R(q, m, t) = \alpha \cdot S_{\text{BM25}}(q, m) + \beta \cdot S_{\text{FAISS}}(q, m) + \gamma \cdot e^{-\lambda \Delta t} \quad (1)$$

Where the components are defined as follows: $S_{\text{BM25}}(q, m)$ represents the normalized BM25 lexical relevance score between the query and the memory text, capturing exact keyword matches critical for code identifiers and error strings. $S_{\text{FAISS}}(q, m)$ represents the normalized cosine similarity between the query embedding and memory embedding from the FAISS HNSW index, capturing semantic relationships even without keyword overlap. $\Delta t = t - t_{\text{last_accessed}}(m)$ represents the elapsed time in hours since the memory was last accessed, implementing a recency bias. λ is the decay constant controlling

memory degradation speed (calibrated at 0.05 per hour for code tasks). α, β, γ are operational weights calibrated to balance lexical against semantic retrieval (recommended: $\alpha = 0.25, \beta = 0.45, \gamma = 0.30$). The exponential decay term ensures that recently accessed memories surface preferentially, preventing stale context from dominating retrieval. This is particularly important for coding workflows where yesterday's debugging session is more relevant than a similar-sounding session from three months ago. The calibration values are derived from empirical evaluation on the LoCoMo long-conversation memory benchmark, where temporal weighting has been shown to improve retrieval accuracy by 12–18% over pure vector search.

4.3 Contradiction Detection Flow

Before committing any record to Semantic Memory, the system executes a cross-check verification to prevent knowledge contamination — the accumulation of conflicting facts that degrades agent reasoning quality over time. This aligns with the hallucination amplification mitigation strategies identified in 2026 production agent research.

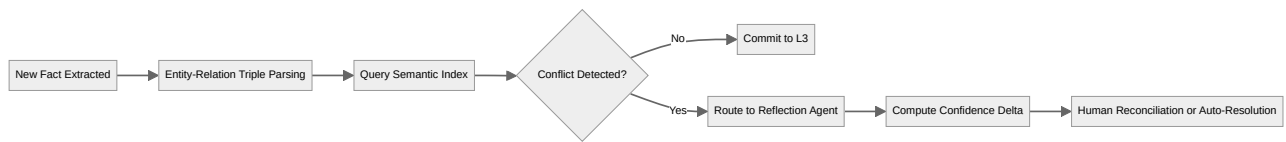


Figure 2: Contradiction Detection Flow for Semantic Memory Writes

4.4 Memory Latency Budget

Production agent systems must maintain interactive response latency. The following table specifies target latencies for each memory operation, based on 2026 benchmarks for local SQLite + FAISS deployments at the 100K vector scale.

Table 6 Memory Operation Latency Budget (100K Vector Scale)

Operation	Target Latency	Backend	Scaling Notes
L1 Working Memory read	< 1 ms	RAM dict	Constant time; no scaling limit
L2 Episodic Memory query (time-filtered)	5–15 ms	SQLite indexed	Sub-linear with proper indexing
L3 Semantic Memory entity lookup	3–8 ms	SQLite indexed	Sub-linear with entity index
FAISS vector search (HNSW, ef=128)	8–20 ms	FAISS HNSW	Logarithmic in vector count
Hybrid re-ranking (BM25 + vector)	15–30 ms	Combined	Parallel execution possible
Context assembly + token budget trim	2–5 ms	Python	Linear in result set size
Total memory overhead	< 80 ms	All layers	Well within interactive budget

5. Deterministic Security Model

5.1 Three-Zone Classification

The Security Gateway implements an unbypassable, deterministic classification structure that categorizes every proposed action into one of three safety zones. Classification is independent of the LLM's confidence or instructions — it operates purely on action payload analysis. This deterministic approach distinguishes the system from probabilistic guardrail frameworks that rely on model-based judges with inherent output variability.

Table 7 Security Zone Classification Matrix

Zone	Danger Level	Permitted Operations	Resolution Gate
GREEN	Safe	Read-only file actions; repository search; code explanation; plan generation; status queries	Auto-execute — pass directly to Executor Agent
YELLOW	Caution	Code editing; local package installation; directory creation; local test execution; git operations	One-click confirmation — UI renders interactive diff preview; 60s timeout auto-cancel
RED	Danger	File deletion; credential modification; environment variable export; network connection; system-level mutation	Explicit typed confirmation — alphanumeric token required; 30s timeout

5.2 Advanced Guardrail Controls

Building on the three-zone foundation, the security model implements five additional guardrail controls that operate at different stages of the execution pipeline. These controls follow defense-in-depth principles established in production AI safety literature: multiple independent checks that make exploitation expensive and detection fast.

PROMPT INJECTION SHIELD

Evaluates all external user inputs and tool outputs using a lightweight local regex classifier combined with a vector blocklist before context synthesis. The regex layer operates in single-digit milliseconds, catching known injection patterns (ignore previous instructions, DAN-mode variants, delimiter confusion). The vector blocklist captures semantically similar but syntactically novel attacks via embedding similarity search against a curated dataset of known prompt injection strategies.

SCOPE LOCKING

Runtime parameters constrain all file system operations to pre-approved root directories declared at session initialization. Commands targeting paths outside the declared scope trigger automatic escalation to RED zone handling, regardless of the operation type. Scope declarations support glob patterns for flexibility (e.g., `/home/user/projects/*`) but never permit traversal above the resolved absolute path. This prevents directory escape attacks where a relative path such as `../../../../etc/passwd` might bypass naive string prefix checks.

FAIL-CLOSED STRATEGY

If an incoming action's security classification matches an ambiguous state (unknown command pattern, parser exception, classifier timeout) the gateway immediately defaults to RED zone handling. The system is never permissive by default. This property is critical for safe operation: an unavailable classifier must not create a window where dangerous actions execute unchallenged. Fail-closed errors are treated as availability incidents with dedicated runbooks.

SECRET SCANNER

All file write payloads pass through high-entropy scanning and token regular expression matching to identify and strip API keys, database connection strings, and private certificates before recording to any persistent storage. The scanner implements the GitHub secret scanning regex library with extensions for common cloud provider credential formats (AWS Access Key ID, GitHub Personal Access Token, OpenAI API Key). Detected secrets are replaced with `<REDACTED:TYPE:HASH>` placeholders that preserve context without exposing the credential.

RATE LIMITING

A maximum of 3 RED-zone actions per session by default, with configurable thresholds. This prevents cascade failures where a single misinterpreted instruction could trigger multiple destructive operations. Rate limit exhaustion triggers mandatory human re-authorization before any additional RED-zone actions are permitted.

5.3 Failure Mode & Effects Analysis (FMEA)

A systematic FMEA identifies potential failure modes across the security pipeline, their effects, detection methods, and mitigation strategies. This analysis follows IEC 60812 standards for system reliability assessment.

Table 8 Security Gateway Failure Mode & Effects Analysis

Failure Mode	Effect	Severity	Likelihood	Detection	Mitigation
Classifier timeout	Action blocked (fail-closed)	Medium	Medium	Timeout alert	Circuit breaker + RED default
False negative (missed dangerous op)	Unauthorized system modification	High	Low	Audit log review	Layered checks + human gating
False positive (safe op flagged RED)	User friction; workflow interruption	Low	Medium	User escalation	Override with typed confirmation
Scope lock bypass via symlink	Directory escape attack	High	Low	Path resolution check	Resolve absolute paths before check
Prompt injection evades regex filter	Malicious instruction execution	High	Low	Vector blocklist	Dual-layer regex + embedding check
Audit log corruption	Tampered history undetected	High	Low	Hash chain validation	Cryptographic SHA-256 chaining
Rate limiter bypass	Cascade destructive operations	High	Low	Counter check	Atomic increment + session binding
Secret scanner false negative	Credential exposure in logs	High	Low	Entropy analysis	High-entropy + regex dual scan

6. Core System Engine Specifications

6.1 Mistake Database Schema

The Mistake Database implements structured incident logging that enables cross-session learning. Each entry captures not only what failed, but the causal chain, remediation applied, and a calibrated confidence adjustment. This structured format supports programmatic querying: "Show me all TypeError incidents in the last 30 days with successful fixes."


```

CREATE TABLE mistake_pool (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp          DATETIME DEFAULT CURRENT_TIMESTAMP,
    task_id            TEXT NOT NULL,
    error_type         TEXT NOT NULL,          -- e.g., 'TypeError',
    'AssertionError', 'Timeout'
    root_cause         TEXT NOT NULL,          -- Human-readable causal analysis
    fix_applied        TEXT NOT NULL,          -- Specific remediation steps taken
    lesson_text        TEXT NOT NULL,          -- Generalized lesson for future
prevention
    confidence_delta   REAL NOT NULL,          -- Post-incident confidence
adjustment [-1.0, 0]
    verification_status TEXT DEFAULT 'pending', -- 'pending', 'verified',
'superseded'
    superseded_by      INTEGER REFERENCES mistake_pool(id), -- If lesson updated
    occurrence_count    INTEGER DEFAULT 1      -- Incremented on repeated similar
errors
);

-- Indexes for query patterns
CREATE INDEX idx_mistake_task ON mistake_pool(task_id);
CREATE INDEX idx_mistake_type ON mistake_pool(error_type);
CREATE INDEX idx_mistake_time ON mistake_pool(timestamp);
CREATE INDEX idx_mistake_verified ON mistake_pool(verification_status) WHERE
verification_status = 'verified';

```

6.2 Cryptographic Audit Log

The Audit Log implements an append-only tamper-evident data structure using cryptographic hash chaining. Each entry's integrity depends on all previous entries, creating a sequential dependency that makes any historical modification immediately detectable. This design mirrors blockchain integrity guarantees while operating within a single SQLite database instance.

```

CREATE TABLE tamper_audit_trail (
    entry_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp          DATETIME DEFAULT CURRENT_TIMESTAMP,
    actor              TEXT NOT NULL,          -- Component or human identity
    action_payload     TEXT NOT NULL,          -- Serialized action description
    security_zone      TEXT NOT NULL CHECK (security_zone IN ('GREEN', 'YELLOW',
'RED')),
    current_hash       TEXT NOT NULL,          -- SHA-256 of this entry
    previous_hash      TEXT NOT NULL          -- SHA-256 of previous entry
(genesis: 0x00...0)
);

```

HASH CHAIN FORMULA

For each audit entry i , the cryptographic hash H_i is computed as:

$$H_i = \text{SHA-256} (H_{i-1} \parallel \text{timestamp}_i \parallel \text{actor}_i \parallel \text{action_payload}_i \parallel \text{security_zone}_i) \quad (2)$$

Where $\parallel$ denotes concatenation, and H_0 (the genesis hash) is defined as a string of 64 zero characters. If any historical entry is altered, all subsequent hash values become invalid, breaking the chain and alerting the security engine. Verification requires only $O(n)$ sequential hash recomputation, making integrity audits computationally feasible even for large log volumes.

6.3 API Contract Specifications

The following table defines the API contracts between core system components. These interfaces are versioned and backward-compatible within major versions to support independent component evolution.

Table 9 Core API Contracts — Request/Response Schemas

Endpoint	Method	Request	Response	Status Codes
/api/v1/plann	POST	{ "goal": string, "context": ContextSnapshot}	{ "task_tree": TaskNode[], "overall_confidence": float}	200, 400, 422, 500
/api/v1/memory/search	POST	{ "query": string, "layers": Layer[], "top_k": int}	{ "results": MemoryEntry[], "scores": float[]}	200, 400, 503
/api/v1/security/classify	POST	{ "action": ActionPayload, "scope": ScopeDecl}	{ "zone": "GREEN YELLOW RED", "confidence": float, "reasoning": string}	200, 400, 500
/api/v1/approval/request	POST	{ "zone": string, "diff": string, "timeout_ms": int}	{ "decision": "approved rejected timeout", "decision_id": UUID}	200, 408 (timeout)
/api/v1/execute	POST	{ "command": string, "sandbox_config": SandboxConfig}	{ "stdout": string, "stderr": string, "exit_code": int, "duration_ms": int}	200, 403 (scope violation), 500
/api/v1/reflect	POST	{ "task_id": UUID, "success": bool, "outputs": ExecutionResult}	{ "lesson_id": int, "confidence_delta": float, "inserted": bool}	200, 201, 500
/api/v1/audit/verify	GET	Query params: from_entry to_entry	{ "valid": bool, "broken_at": int null, "computed_hash": string}	200
/api/v1/rollback	POST	{ "task_id": UUID, "strategy": "git-stash snapshot"}	{ "restored": bool, "pre_state_hash": string, "post_verify": bool}	200, 404, 500

7. Testing & Verification Strategy

7.1 Testing Pyramid

A production-grade agent system requires testing at four levels: unit tests for individual components, integration tests for pipeline stages, end-to-end tests for complete workflows, and property-based tests for security invariants. The following matrix defines test coverage targets.

Table 10 Testing Strategy — Coverage Targets

Test Level	Scope	Target Coverage	Tools	CI Trigger
Unit Tests	Individual functions, security classifiers, memory operations	> 85%	pytest, pytest-cov	Every commit
Integration Tests	Component interactions (Memory Agent → SQLite → FAISS)	> 70%	pytest, Docker Compose	Every PR
E2E Tests	Complete pipeline execution (input → output)	> 50%	pytest + Playwright	Daily + pre-release
Security Tests	Injection attempts, scope bypasses, secret leakage	100% of known patterns	PyRIT, custom probes	Weekly
Chaos Tests	Component failure, network partition, resource exhaustion	All critical paths	chaostoolkit	Weekly
Performance Tests	Latency, throughput, memory usage at scale	All latency budgets	locust, pytest-benchmark	Pre-release

7.2 Security Testing Protocol

The security testing protocol implements the MITRE ATLAS framework for adversarial AI testing. Test cases cover four attack categories: prompt injection (direct, indirect, multi-turn), tool misuse (scope bypass, privilege escalation), data exfiltration (secret extraction via side channels), and availability attacks (resource exhaustion, cascade trigger).

Algorithm: Adversarial Test Execution

Input: Agent system under test S , attack dataset $D_{attacks}$, baseline dataset $D_{baseline}$

Output: Security score $Score(S) \in [0, 100]$

For each attack $a \in D_{attacks}$:

1. Execute a against S with 3 independent runs
2. Record: blocked (security gateway), caught (verifier), succeeded, or false negative
3. Calculate per-attack score: $score(a) = 100$ if blocked, 50 if caught, 0 if succeeded

$$Score(S) = \frac{1}{|D_{attacks}|} \sum_{a \in D_{attacks}} score(a)$$

Threshold: $Score(S) \geq 85$ for production deployment authorization.

7.3 Chaos Engineering Plan

Table 11 Chaos Engineering Experiments

Experiment	Target Component	Injection	Expected Behavior	Abort Condition
Memory Agent latency spike	FAISS query	500ms delay on 30% of queries	Degrade to BM25-only; log warning	> 5 consecutive failures
SQLite corruption	Episodic DB	Read-only lock on SQLite file	Queue writes; alert operator	Data loss risk
Security gateway timeout	Zone classifier	Classifier response > 5s	Fail-closed → RED zone	No unclassified actions
Human approval timeout	Approval Engine	Simulate no response for 120s	Auto-cancel; rollback if YELLOW/RED	No orphaned operations
Subprocess memory leak	Executor Agent	Runaway memory allocation	cgroup OOM kill; clean restart	Host memory < 80%
Audit log disk full	Audit Logger	Simulate ENOSPC on log volume	Critical alert; halt non-read ops	No silent write failures

8. Deployment Architecture & Observability

8.1 Deployment Topology

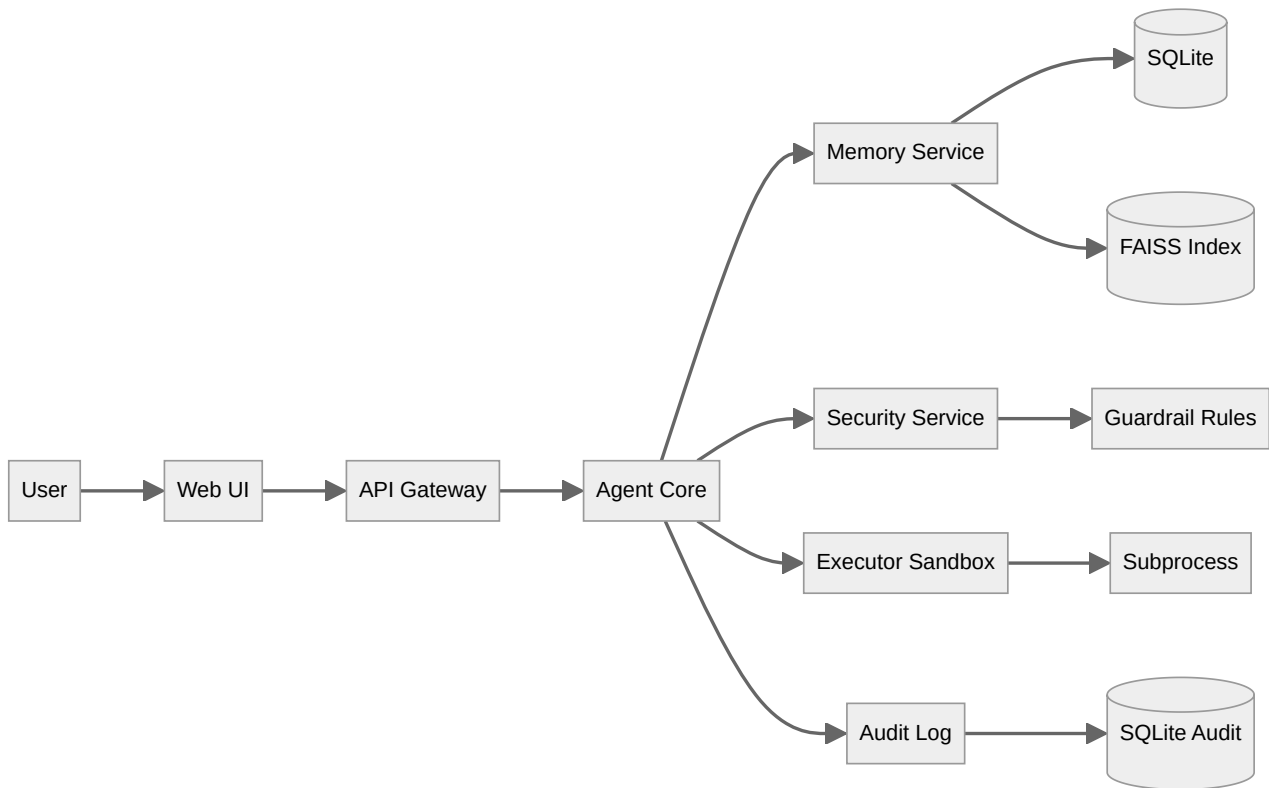


Figure 3: Deployment Topology — Service Boundaries

The recommended deployment topology uses Docker Compose for development and single-node production, with clear service boundaries enabling future migration to Kubernetes. The Agent Core orchestrates all pipeline stages while the Memory Service, Security Service, and Executor Sandbox operate as independent containers with defined API contracts. This separation supports independent scaling: the Executor Sandbox can be replicated for parallel task execution while the Security Service remains a singleton to maintain consistent state.

8.2 Performance & Scalability Specifications

Table 12 Performance Targets — Single-Node Deployment

Metric	Target	Measurement Method	Alert Threshold
End-to-end pipeline latency (GREEN)	< 3s	API response time	p95 > 5s
Memory retrieval latency	< 80ms	FAISS + BM25 benchmark	p99 > 200ms
Security classification latency	< 50ms	Gateway benchmark	p99 > 150ms
Audit log write throughput	> 1000 writes/s	SQLite WAL benchmark	< 500/s
Concurrent task execution	4 parallel	Executor pool test	> 6 queued
Memory usage at 100K vectors	< 2GB	psutil monitoring	> 3.5GB
SQLite database size growth	< 100MB/week	Filesystem monitoring	> 200MB/week

8.3 Monitoring & Alerting Framework

Production observability uses a three-pillar approach: metrics (Prometheus), logs (structured JSON), and traces (OpenTelemetry span context). All security events, memory operations, and pipeline stage transitions emit structured telemetry with correlation IDs for end-to-end request tracing.

Table 13 Monitoring & Alerting Configuration

Signal	Metric / Log Pattern	Alert Rule	Severity	Response
Security gateway fail-closed rate	<code>security_fallback_total / security_total > 0.05</code>	5-minute rate	Critical	Page on-call; investigate classifier
RED zone action rejected	<code>approval_rejected_total</code> spike	> 3 in 5 minutes	Warning	Review rejection patterns
Hash chain integrity	<code>audit_verify_valid == false</code>	Immediate	Critical	Halt operations; forensic analysis
Memory retrieval latency	<code>memory_search_duration_seconds p99 > 0.2</code>	Sustained 10 minutes	Warning	Scale FAISS resources
Executor sandbox OOM	<code>executor_oom_total</code>	> 1 in 1 hour	Warning	Review cgroup limits
Secret scanner detection	<code>secret_detected_total</code>	Any detection	Critical	Immediate alert; rotate credential
Reflection agent failure	<code>reflection_error_total</code>	> 5 in 1 hour	Info	Review LLM API health

9. Trust Principles for Industrial Development

The following trust principles are not aspirational guidelines but architectural invariants — properties that must hold at every stage of system operation. They are derived from production safety engineering practices and represent commitments that can be verified through testing and audit.

Table 14 Trust Principles — Verification Methods

Principle	Statement	Verification Method
Trust Evidence, Not the Model	Never assume a command succeeded because the LLM states it did. Validate via system calls, diff checks, and deterministic compiler output.	Verifier layer test assertions on every execution; require exit code 0 + output match
Immutable Audit Trail	Every interaction is logged sequentially. The hash chain prevents undetected history alteration.	Cryptographic audit verification on startup and on demand; chain break detection
Pre-Action Snapshots	Before every YELLOW/RED modification, capture file state or git-stash to enable deterministic rollback.	Snapshot existence verified before approval; rollback test in CI
No Secret Persistence	API keys and credentials are never written to disk or standard logs. Stored only in volatile environment variables.	Secret scanner on all writes; grep audit for credential patterns in CI
Explainability First	Every Planner decision includes human-readable rationale before execution.	Plan output schema requires reasoning field; rejection if missing
Confidence Threshold Gating	Steps below 0.72 confidence trigger human review regardless of zone classification.	Confidence filter unit tests at boundary (0.719 → escalate, 0.72 → pass)
Scope Declaration	Each session declares allowed directories/packages at start; out-of-scope auto-escalates to RED.	Scope resolution test for path traversal, symlink, and relative path variants
Fail Closed	Security Gateway defaults to RED on uncertain classification. Never permissive.	Chaos test: classifier timeout must produce RED, not GREEN

10. Risk Assessment Matrix

A comprehensive risk assessment evaluates architectural, implementation, and operational risks across five dimensions: Security, Scalability, Complexity, Dependency, and Schedule. Each risk is rated by impact (1–5) and probability (1–5), with risk score = impact × probability. Risks scoring ≥ 15 require mitigation plans before proceeding to the next development phase.

Table 15 Risk Assessment Matrix

Risk ID	Description	Category	Impact	Probability	Score	Mitigation
R1	Security Gateway implementation delay blocks recruiter demo	Schedule	5	4	20	Implement minimal 3-zone classifier first; enhance later
R2	FAISS index corruption on unclean shutdown	Technical	4	3	12	WAL + periodic index snapshots; auto-rebuild from SQLite
R3	Scope too ambitious for internship timeline; demo instability	Schedule	4	4	16	Freeze features after Phase 2; polish before expanding
R4	Local LLM capability insufficient for complex reasoning tasks	Technical	3	4	12	Fallback to API mode; benchmark tasks against local model first
R5	Prompt injection evades dual-layer shield	Security	5	2	10	Continuous red-teaming; update blocklist weekly
R6	SQLite performance degradation at >1M entries	Scalability	3	3	9	Partition by month; archive old data; add connection pooling
R7	Reflection Agent produces incorrect root cause analysis	Technical	3	3	9	Human verification for new lesson types; confidence threshold
R8	Dependency on Ollama/Whisper with incompatible updates	Dependency	3	3	9	Pin versions; test in CI before upgrade
R9	Recruiter demo fails due to environment-specific issue	Operational	4	3	12	Containerized demo environment; rehearse on target hardware
R10	Audit log disk exhaustion causes system halt	Operational	3	2	6	Log rotation with compression; 30-day retention policy

11. Engineering Skills Matrix

The following matrix maps each architectural component to the engineering competencies it demonstrates. This matrix serves as a recruiter evaluation aid, providing explicit connections between system components and demonstrable skills.

Table 16 Engineering Competency Mapping

Component	Primary Skills	Demonstrated Via
Security Gateway (3-zone classifier)	Security engineering, defense-in-depth, deterministic systems	Fail-closed design; FMEA analysis; scope-locking implementation
Memory Architecture (4-layer + FAISS)	Distributed systems, database design, information retrieval	Hybrid BM25+FAISS retrieval; decay scoring formula; SQLite schema design
Audit Logger (SHA-256 chain)	Cryptography, tamper-evident systems, compliance engineering	Hash chain formula; integrity verification API; append-only guarantees
Reflection Agent + Mistake DB	ML systems, feedback loops, structured data engineering	Schema design; confidence delta calibration; cross-session learning
API Contracts (8 endpoints)	API design, system interfaces, versioning strategy	RESTful schema design; status code selection; backward compatibility
Testing Strategy (6 levels)	Quality assurance, chaos engineering, security testing	Pyramid coverage targets; MITRE ATLAS adversarial tests; chaos experiments
Deployment Architecture	DevOps, containerization, observability	Docker Compose topology; Prometheus alerts; OpenTelemetry traces
Performance Specifications	Systems performance, capacity planning, SLO engineering	Latency budgets; scaling thresholds; alert rules with severities

12. Recruiter Showcase & Evaluation Scenario

12.1 Demo Script Timeline

The two-minute demonstration is designed to communicate engineering depth and architectural maturity through five sequential segments, each building on the previous to tell a cohesive story of supervised AI system design.



Figure 4: Two-Minute Demo Script Flow

Table 17 Demo Script — Detailed Timeline

Time	Segment	Action	What Recruiter Sees
0:00–0:30	Core Architecture	Present the system topology diagram. One sentence per pipeline stage.	System design depth; understanding of multi-agent orchestration; security-first thinking
0:30–0:50	Guardrail Validation	Trigger a YELLOW zone operation (source code edit). Show interactive diff preview holding execution pending human authorization.	Human-in-the-loop implementation; safety before speed; diff visualization capability
0:50–1:10	Error Interception	Intentionally inject a syntax exception or failing test. Show Verifier isolating the execution fault and halting the pipeline.	Defensive programming; test-driven verification; graceful failure handling
1:10–1:30	Automated Reflection	Show Reflection Agent parsing the runtime error, appending structured entry to Mistake DB, and initiating git-stash rollback.	Self-improving systems; structured learning from failure; automated recovery
1:30–2:00	Cryptographic Integrity	Query the SQLite Audit Log. Demonstrate how altering historical data breaks SHA-256 verification chain.	Cryptography application; tamper-evident design; compliance-grade audit capability

12.2 Talking Points by Competency

For each segment of the demo, the following talking points map demonstrated capabilities to specific engineering competencies that recruiters evaluate.

ARCHITECTURE SEGMENT (0:00–0:30)

Opening line: "I built a supervised AI Operating System, not a chatbot."

Key points: (1) No component bypasses the Security Gateway — this is kernel-level isolation, not middleware filtering. (2) The execution pipeline is linear and deterministic: same input always produces the same zone classification. (3) Human authority is embedded at the architecture level, not added as an afterthought.

SECURITY SEGMENT (0:30–0:50)

Opening line: "The AI proposes; the human approves."

Key points: (1) The Security Gateway uses deterministic classification, not model-based judgment — eliminating the probabilistic failure mode where an AI judge incorrectly permits a dangerous action. (2) Scope locking prevents directory escape attacks via symlink resolution and absolute path canonicalization. (3) Fail-closed means safety is the default, not a configuration.

REFLECTION SEGMENT (1:10–1:30)

Opening line: "The system learns from its mistakes structurally, not just statistically."

Key points: (1) The Mistake DB captures root cause and fix, not just error logs, enabling cross-session retrieval: "Have we seen this TypeError before?" (2) Confidence deltas calibrate the Planner's future estimates based on actual outcomes. (3) The rollback engine provides deterministic recovery, not best-effort restoration.

13. Development Roadmap

Table 18 Five-Phase Development Roadmap

Phase	Name	Deliverables	Status	Target
1	Foundation	LLM integration, code generation, live preview, terminal access, file editing, user permission flow	COMPLETE	Q4 2025
2	Memory & Reflection	SQLite memory layer, reflection agent, mistake database, audit log + hash chain, hybrid BM25+FAISS retrieval	ACTIVE	Q2 2026
3	Intelligence Layer	FAISS vector memory, project knowledge graph, confidence scoring, auto-verification pipeline, contradiction detection	NEXT	Q3 2026
4	Security & Voice	Security agent + gateway, voice interface (Whisper), rollback engine, prompt injection shield, multi-agent review	PLANNED	Q4 2026
5	MVP & Showcase	Personal AI OS MVP, architecture diagrams, 2-minute demo video, daily usage testing, internship portfolio	PLANNED	Q1 2027

Critical recommendation: Based on the risk assessment (R3, score 16), freeze feature expansion after Phase 2 completion and invest in polish, stability testing, and demo reliability before proceeding to Phases 3–4. A reliable two-minute demo of memory + security + rollback decisively outperforms an unstable ten-feature

demonstration. The Security Gateway and Reflection Agent are the highest-priority implementations ahead of voice and vector memory enhancement.

References

1. Packer, G., et al. (2024). *MemGPT: Towards LLMs as Operating Systems*. University of California, Berkeley. arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
2. Zhou, Y., et al. (2024). *Reinforce LLM Reasoning through Multi-Agent Reflection*. arXiv:2506.08379. <https://arxiv.org/abs/2506.08379>
3. Shinn, N., et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. *NeurIPS 2023*. <https://arxiv.org/abs/2303.11366>
4. Coralogix. (2026). *What Are AI Guardrails? A Guide for Production LLMs*. Coralogix AI Blog. Retrieved May 2026. <https://coralogix.com/ai-blog/ai-guardrails/>
5. Digital Applied. (2026). *AI Agent Memory 2026: Vector, Graph, Episodic Update*. <https://www.digitalapplied.com/blog/ai-agent-memory-vector-graph-episodic-2026>
6. RankSquire. (2026). *Vector Memory Architecture For AI Agents — 2026 Blueprint*. Retrieved March 2026. <https://ranksquire.com/2026/03/12/vector-memory-architecture-for-ai-agents-2026/>
7. Zylos Research. (2026). *Agent Self-Correction: From Reflexion to Process Reward Models*. Retrieved May 2026. <https://zylos.ai/research/2026-05-12-agent-self-correction-reflexion-to-prm>
8. Chen, J., et al. (2024). *Reflection-Reinforced Self-Training for Language Agents*. *EMNLP 2024*. ACL Anthology. <https://aclanthology.org/2024.emnlp-main.861.pdf>
9. FERZ. (2026). *FERZ is not Guardrails: Architectural Comparison*. FERZ Governance. <https://ferz.ai/governance/comparisons/ferz-is-not-guardrails>
10. GeeksforGeeks. (2025). *Episodic Memory in AI Agents*. Retrieved August 2025. <https://www.geeksforgeeks.org/artificial-intelligence/episodic-memory-in-ai-agents/>